

SMART HOSPITAL MANAGEMENT SYSTEM

A Desktop-Based Hospital Management Solution Developed in C# and SQL Server

Introduction:

The Smart Hospital Management System is a desktop-based application developed using C# Windows Forms and SQL Server. The main purpose of this project is to help hospitals manage their daily operations in a simple and organized way. In many small hospitals and clinics, patient records, appointments, billing, and reports are often handled manually, which can lead to mistakes, data loss, and difficulty in managing information.

This system was designed to reduce manual work by providing an easy-to-use graphical interface where users can manage hospital activities digitally. The project includes modules for patient management, appointment handling, billing management, report generation, and dashboard statistics. SQL Server database connectivity is used to store and retrieve records securely.

The system also provides charts, search functionality, role-based login access, and data visualization features to improve the user experience. The project was developed by applying concepts of Visual Programming, database connectivity, modular programming, and object-oriented programming.

Problem Statement:

Many hospitals and clinics still rely on manual methods for managing patient records, appointments, and billing information. Manual systems consume time, increase paperwork, and create chances of human errors such as duplicate records, missing information, and incorrect billing calculations.

Managing hospital data without a proper computerized system also makes searching, updating, and generating reports difficult. Staff members may face problems in handling large amounts of data efficiently.

The purpose of this project is to develop a Smart Hospital Management System that provides a simple and efficient solution for managing hospital operations digitally. The

system helps users store records securely, perform billing calculations automatically, manage appointments, and generate reports through an easy graphical interface.

Objectives:

The main objectives of the Smart Hospital Management System are:

- To develop a desktop-based hospital management application using C# and SQL Server.
- To manage patient records efficiently through add, update, delete, search, and view operations.
- To handle appointment scheduling between patients and doctors.
- To generate hospital bills automatically with consultation, medicine, lab test, and tax calculations.
- To display hospital statistics and revenue information using charts and reports.
- To provide a role-based login system for different users.
- To reduce manual paperwork and improve data management efficiency.
- To apply concepts of Visual Programming, database connectivity, and object-oriented programming in a real-world project.

System Design:

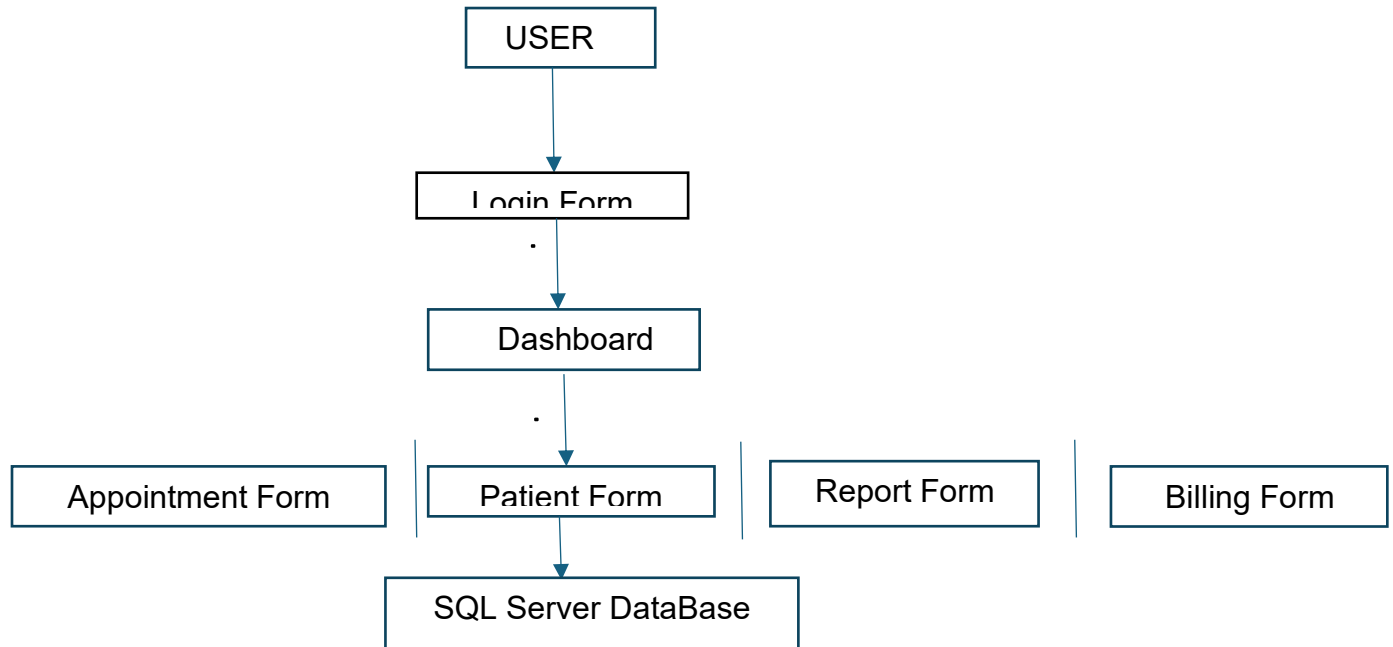
This system consists of following modules that are properly structured and organized and describes how data flows:

- **Login Module**
User enters username, password, and role. After successful login, the dashboard opens.
- **Dashboard Module**
It works as the main menu and connects all forms: Patient, Appointment, Billing, and Reports.
- **Patient Management Module**
Used to add, update, delete, search, and view patient records.
- **Appointment Management Module**
Used to schedule, update, cancel, search, and view appointments.
- **Billing Module**
Used to calculate bills, add bill records, update, delete, search, and view billing records.

- **Reports Module**

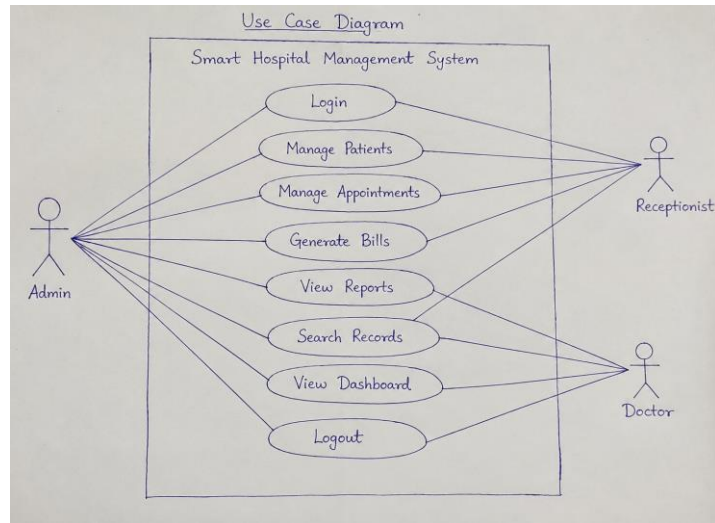
Shows total patients, doctors, appointments, revenue, charts, and records in DataGrid View.

Simple architecture diagram of this system should be:



Use Case Diagram:

This project can have a following use case diagram that best describes all the characters involved in the system and what kind of tasks they are managing like schedule appointment, generate bill, view reports etc



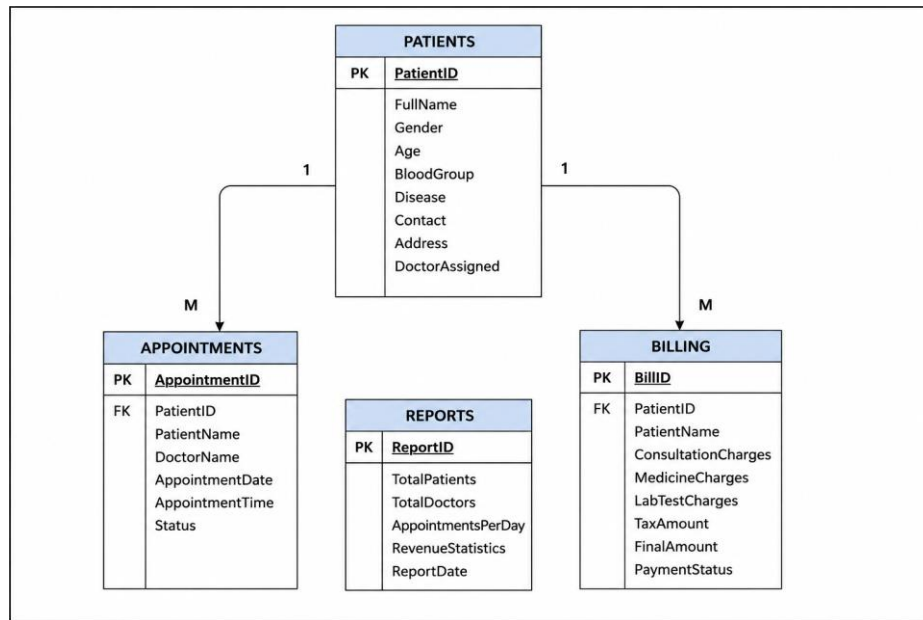
Explanation:

This use case diagram represents the main functionalities of the Smart Hospital Management System and the users who interact with it. The system has three main actors: Admin, Receptionist, and Doctor.

The admin has full access to all system features such as managing patients, appointments, bills, reports, records, dashboard, and logout. The Receptionist can handle patient records, appointments, billing, and reports. The Doctor can view reports, search records, access the dashboard, and logout from the system.

The diagram shows how different users interact with different modules of the hospital management system according to their responsibilities.

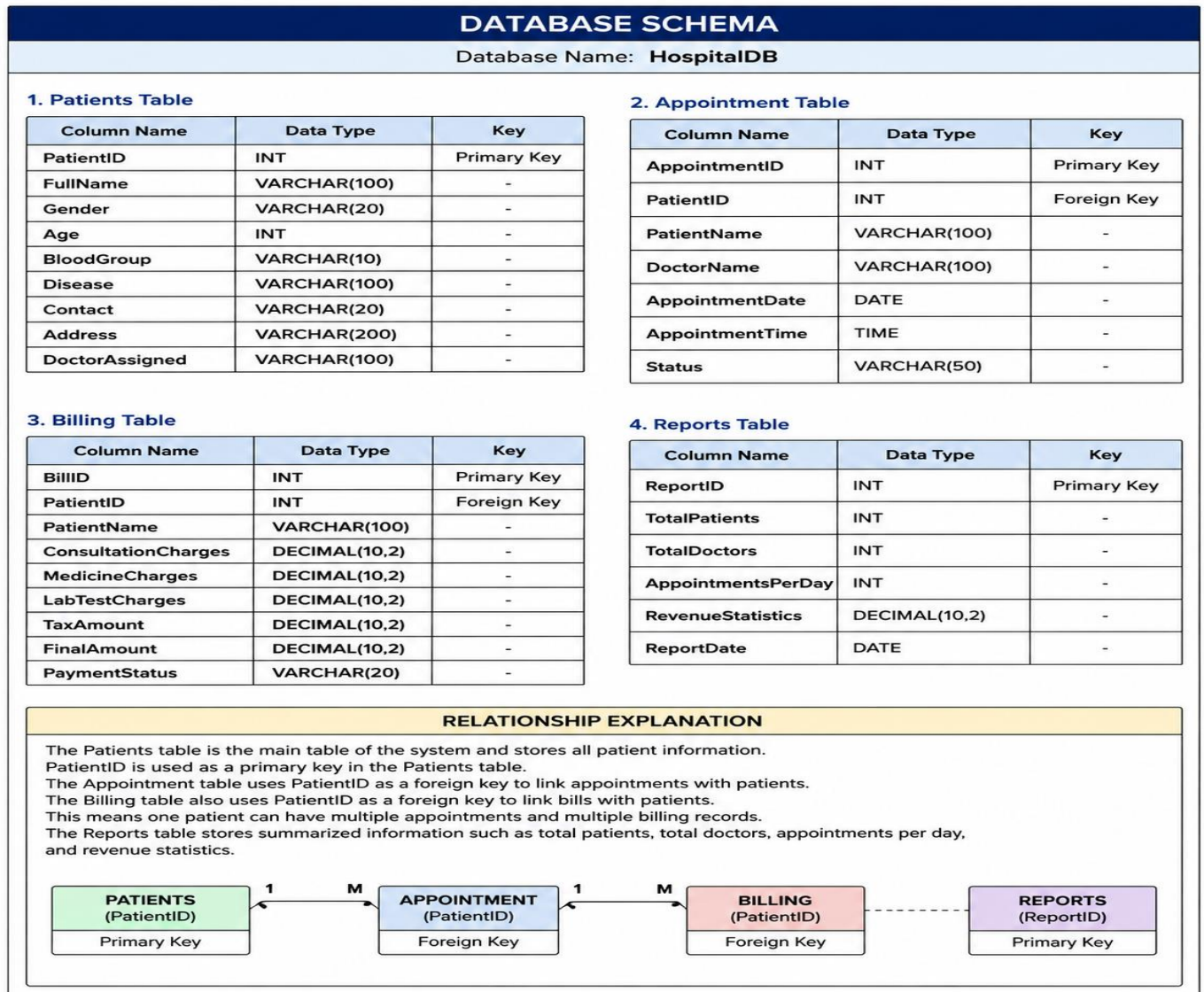
ER Diagram:



Explanation:

This ER diagram represents the database structure of the Smart Hospital Management System. The Patient's table is the main entity that stores patient information such as name, age, disease, contact, and assigned doctor. A patient can have multiple appointments and billing records, which creates a one-to-many relationship between Patients, Appointments, and Billing tables. The Appointments table stores appointment details, while the Billing table manages patient bill information including charges and payment status. The Reports table stores hospital statistics such as total patients, appointments, doctors, and revenue details.

Database Schema:



GUI Screenshots:

Login form:

A screenshot of a 'Login Setup' window for a 'SMART HOSPITAL MANAGEMENT SYSTEM'. The window is titled 'Login Setup' and has a light gray background. It contains three input fields: 'Username' with the value 'admin', 'Password' with masked characters '***', and 'Role' with a dropdown menu showing 'Admin'. Below these fields are two buttons: 'Login' and 'Exit'. The window is running in a Visual Studio environment, with the 'Developer PowerShell', 'Error List', 'Task List', and 'Output' panes visible at the bottom. The Windows taskbar is also visible at the very bottom, showing the time as 9:39 PM on 5/24/2026.

This form is used for user authentication. The user enters username, password, and selects a role such as Admin, Doctor, or Receptionist to access the system securely.

Dashboard Form:

A screenshot of a 'Dashboard' window for a 'Patient Management System'. The window is titled 'Dashboard' and has a light gray background. It features a sidebar on the left with five buttons: 'Patient', 'Appointment', 'Billings', 'Report', and 'Logout'. The main area of the window is titled 'Patient Management System' and is currently empty. The window is running in a Visual Studio environment, with the 'Developer PowerShell', 'Error List', 'Task List', and 'Output' panes visible at the bottom.

The dashboard acts as the main navigation panel of the system. It provides quick access to Patient Management, Appointment Management, Billing, Reports, and Logout options.

Patient Management Form:

The PatientForm application window contains the following fields and controls:

- Buttons:** Add, Updat, Delete, Search, Clea, Back, View.
- Fields:**
 - PatientID: 5
 - Full Name: Ahmed
 - Contact: 0312665476
 - Age: 23
 - Disease: covid
 - Address: Faisalabad
 - Doctor Assigned: Dr Ahmed
 - Gender: A- (dropdown)
 - Sex: Male (dropdown)
- Table:**

	FullName	Gr
▶	Ali	Ma

This form is used to manage patient records. The user can add, update, delete, search, and view patient information stored in the SQL Server database.

Billing Form:

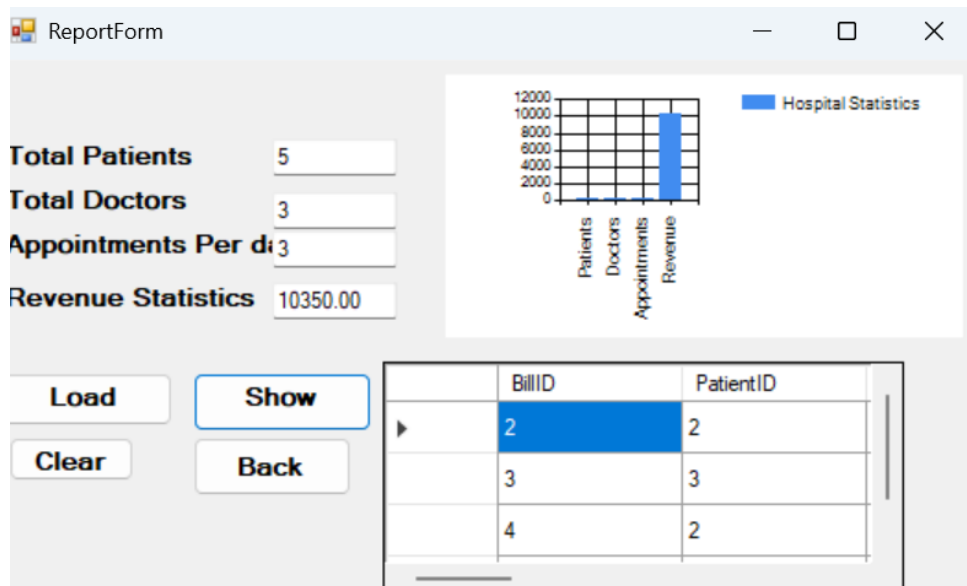
The BillingSystem application window contains the following fields and controls:

- Buttons:** Add, Updat, Delet, Searc, Clear, View, Bac, Calculate, Print.
- Fields:**
 - Bill ID: 4
 - Patient ID: 2
 - Patient Name: Ahsan
 - Consultation Charge: 1500
 - Medicine Charges: 500
 - Lab Test Charges: 350
 - Tax Amount: 200
 - Final Amount: 4000
 - Payment Status: Pending (dropdown)
- Table:**

	BillID	PatientID
▶	2	2
	3	3
	4	2

The billing form is used to generate hospital bills. It calculates consultation charges, medicine charges, lab test charges, tax amount, and final bill amount along with payment status.

Report Form:



This form displays hospital statistics and reports. It shows total patients, doctors, appointments, revenue statistics, charts, and records using DataGridView and graphical visualization.

Code Explanation:

Login Form Code Explanation:

The Login Form is used for authentication.

The system checks the username, password, and selected role. If the entered information is correct, the user is redirected to the Dashboard form. Otherwise, an error message is displayed.

Main concepts used:

- If-Else Conditions
- ComboBox
- MessageBox
- Form Navigation

Patient Management Code Explanation:

This form performs CRUD operations (Create, Read, Update, Delete) for patient records using SQL Server database connection.

Functions implemented:

- Add Patient
- Update Patient
- Delete Patient
- Search Patient
- View Records

Important concepts:

- SqlConnection
- SqlCommand
- DataGridView
- SQL Queries

Example:

```
cmd.ExecuteNonQuery();
```

This line executes INSERT, UPDATE, or DELETE SQL commands in the database.

Appointment Form Code Explanation:

The Appointment Form is used to schedule patient appointments with doctors. The form stores appointment date, time, patient details, and appointment status.

Controls used:

- DateTimePicker
- ComboBox
- DataGridView

Database operations:

- Insert Appointment
- Search Appointment
- Cancel Appointment

Billing Form Code Explanation;

The Billing Form calculates hospital charges and stores billing information in the database.

The system calculates:

- Consultation Charges

- Medicine Charges
- Lab Test Charges
- Tax Amount
- Final Amount

Example:

```
double total = consultation + medicine + lab + tax;
```

This code calculates the final bill amount.

Report Form Code Explanation:

The Report Form displays hospital statistics using charts and DataGridView.

Features included:

- Total Patients
- Total Doctors
- Revenue Statistics
- Graphical Reports

Controls used:

- Chart Control
- DataGridView
- TextBoxes

. Database Connectivity Explanation:

The project uses SQL Server LocalDB for database storage.

Connection string used:

```
SqlConnection con = new SqlConnection(@"Data
Source=(localdb)\MSSQLLocalDB;Initial Catalog=HospitalDB;Integrated
Security=True");
```

Exception Handling Explanation:

Try-Catch blocks are used to handle runtime errors and prevent the application from crashing.

Example:

```

try
{
    con.Open();
}
catch(Exception ex)
{
    MessageBox.Show(ex.Message);
}

```

This improves the reliability of the system.

Testing & Results:

Login Module Testing

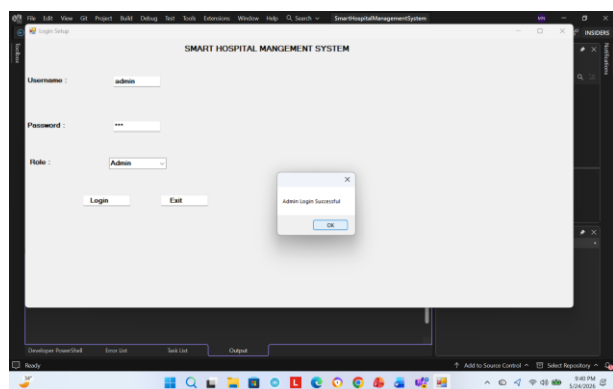
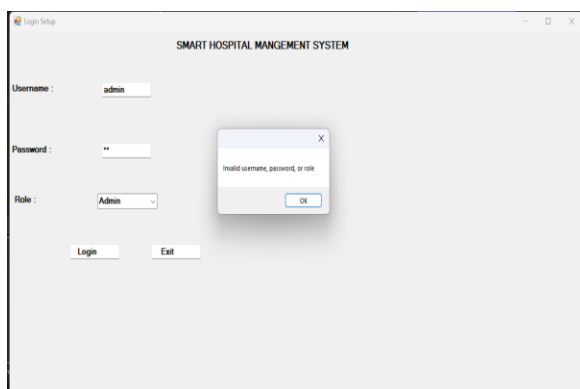
Successful Login

This shows successful admin authentication. When the correct username, password, and role are entered, the system displays the message **“Admin Login Successful”** and allows access to the dashboard.

Invalid Login

This shows system validation for incorrect login credentials.

When invalid username, password, or role is entered, the system displays an error message and prevents unauthorized access.



Patient Management Module Testing

Add Patient Record

This screenshot shows successful insertion of patient data into the database.

The system stores patient information such as Patient ID, Name, Disease, Address, Blood Group, and Gender successfully.

The screenshot displays the 'PatientForm' application window. The form contains the following fields and values:

- PatientID: 3
- Full Name: ahsan
- Contact: 123457
- Age: 23
- Disease: fever
- Address: johar town
- Doctor Assigned: dr ahsan
- Blood Group: A+ (selected from a dropdown)
- Gender: Male (selected from a dropdown)

Buttons visible include 'Add', 'Update', 'Delete', 'Search', 'Clear', 'Back', and 'View'. A small table at the bottom shows a single row with PatientID 3 and Full Name ahsan. A modal dialog box in the foreground displays the message 'Patient Added Successfully' with an 'OK' button.

Update Patient Record

This screenshot shows successful updating of an existing patient record.

The system modifies patient information and updates the database correctly.

The screenshot displays the 'PatientForm' application window with the same data as the previous screenshot. The 'Update' button is highlighted in blue. A modal dialog box in the foreground displays the message 'Patient Record Updated' with an 'OK' button.

Delete Patient Record

This screenshot shows successful deletion of a patient record.

The selected patient information is removed successfully from the database.

The screenshot shows a Windows application titled "PatientForm". It contains several text input fields for patient information: PatientID (3), Full Name (ahsan), Contact (123457), Age (23), Disease (fever), and Address (johar town). There are also dropdown menus for "Doctor Assigned" (dr ahsan), "A+", and "Male". To the right of these fields are buttons: "Add", "Update", "Delete", "Search", "Clear", "Back", and "View". A small table is visible below the input fields, showing columns for "PatientID" and "Full Name". A modal dialog box titled "Patient Deleted" with an "OK" button is overlaid on the right side of the form.

Appointment Module Testing

Schedule Appointment

This screenshot shows successful appointment scheduling between patient and doctor.

The system stores appointment date, time, and doctor details correctly.

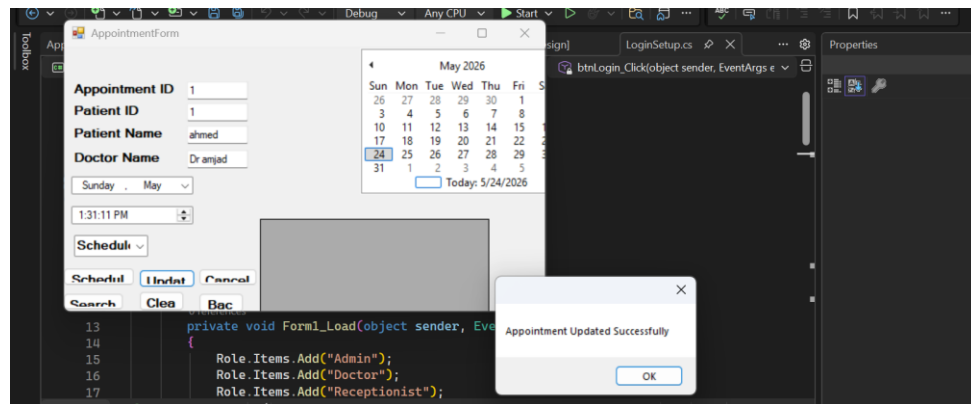
The screenshot shows a Windows application titled "AppointmentForm". It contains several text input fields: Appointment ID (1), Patient ID (1), Patient Name (ahmed), and Doctor Name (Dr amjad). There are also dropdown menus for "Sunday" and "May", and a time input field showing "1:31:11 PM". Below these fields are buttons: "Schedule", "Update", "Cancel", "Search", "Clear", and "Back". A calendar widget is displayed, showing the month of May 2026, with the date 24 selected. A modal dialog box titled "Appointment Scheduled Successfully" with an "OK" button is overlaid on the right side of the form. In the background, a code editor window is visible, showing the following C# code:

```
private void Form1_Load(object sender, EventArgs e)
{
    Role.Items.Add("Admin");
    Role.Items.Add("Doctor");
    Role.Items.Add("Receptionist");
}
```

Update Appointment:

This screenshot shows successful updating of appointment details.

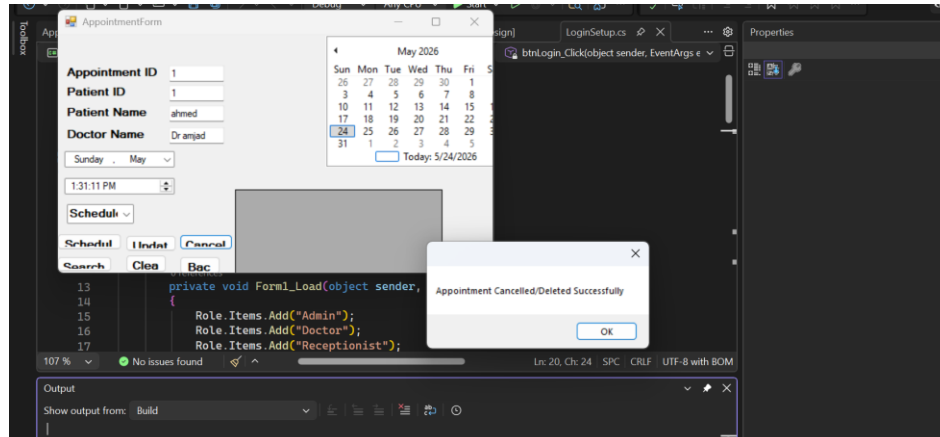
The system updates appointment information such as doctor name, date, or timing successfully.



Cancel/Delete Appointment

This screenshot shows successful cancellation or deletion of an appointment record.

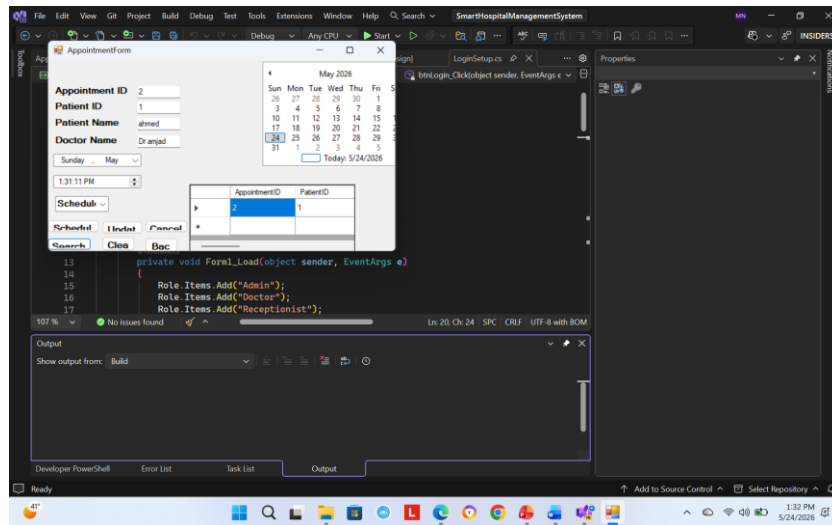
The system removes the selected appointment from the database successfully.



View Appointment Records

This screenshot shows appointment records displayed in the DataGridView.

The system retrieves appointment information correctly from the database.

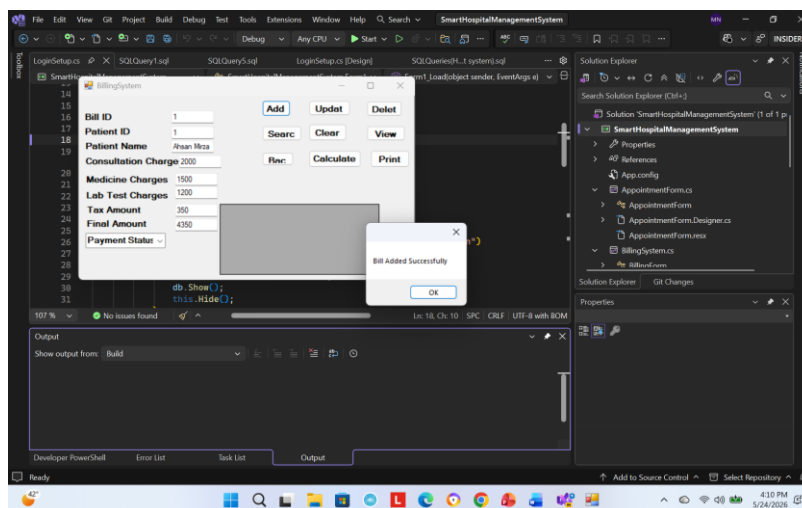


Billing Module Testing

Add Billing Record

This screenshot shows successful generation of a patient bill.

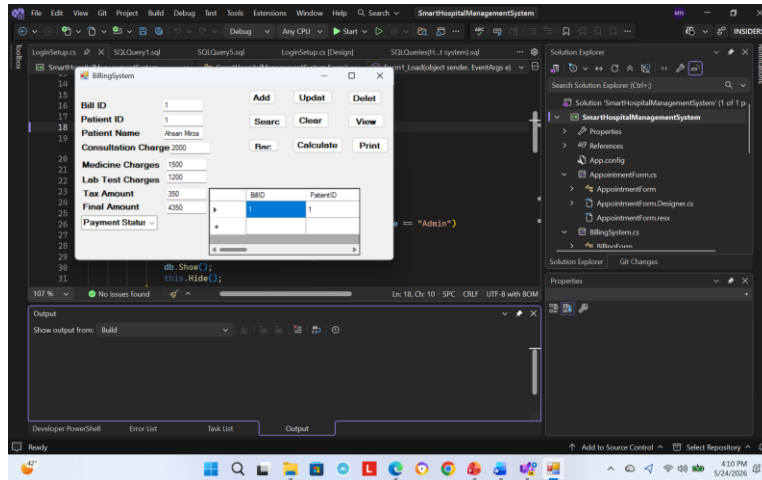
The system calculates consultation, medicine, lab test, tax, and final amount successfully.



View Billing Records

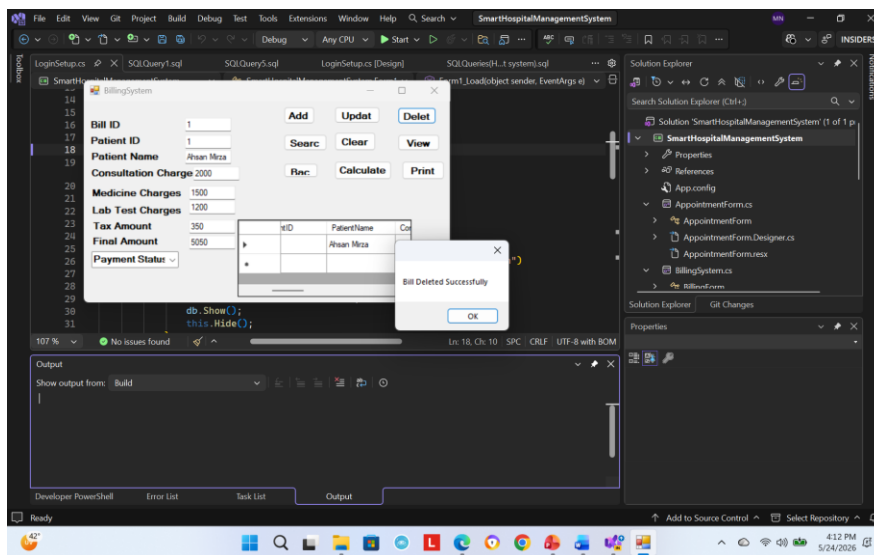
This screenshot shows stored billing records displayed in tabular form.

The system retrieves billing information successfully from the database.



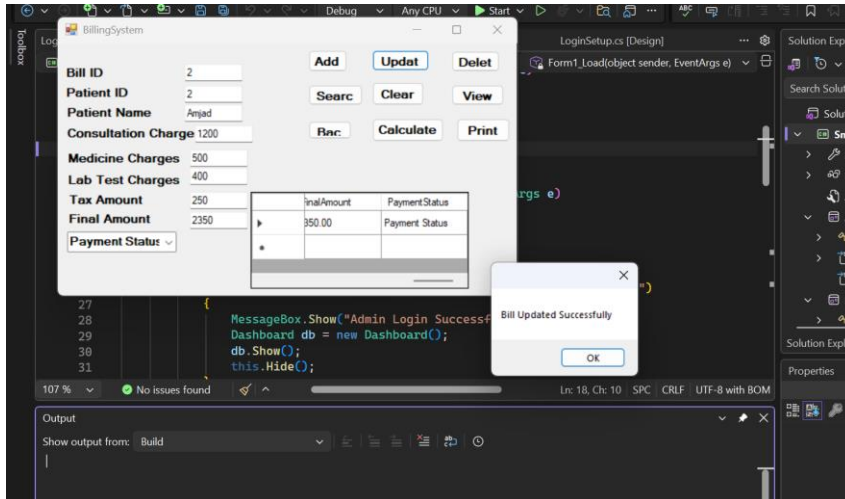
Delete Billing Record

This screenshot shows successful deletion of a billing record from the system database.



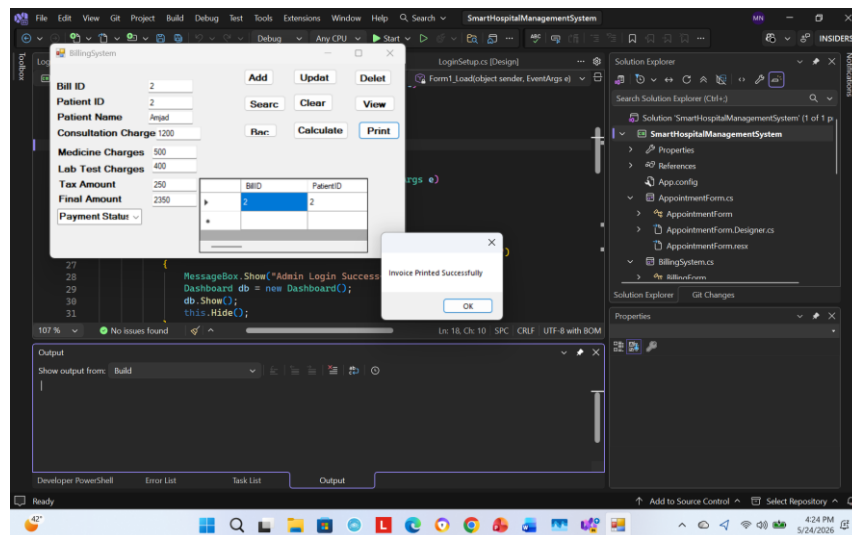
Update Billing Record

This screenshot shows successful updating of billing information and recalculation of total charges.



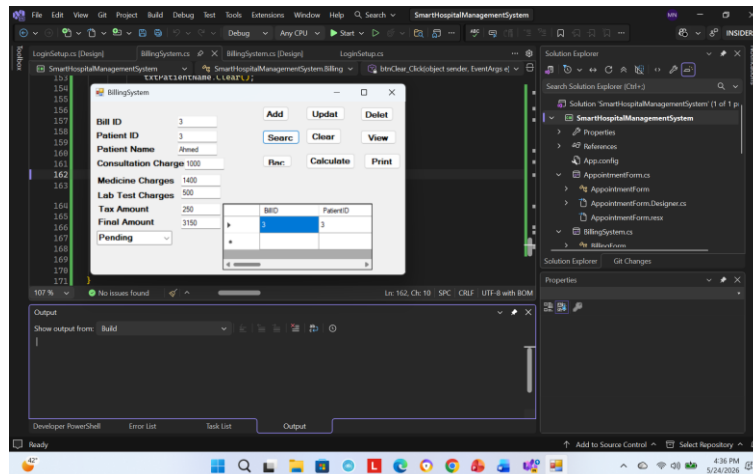
Print Invoice

This screenshot shows successful invoice printing functionality of the billing module.



Search Billing Record

This screenshot shows successful searching and retrieval of billing information using Bill ID or Patient ID.



Report Module Testing

Load Report

This screenshot shows successful loading of hospital statistical data including total patients, doctors, appointments, and revenue statistics.

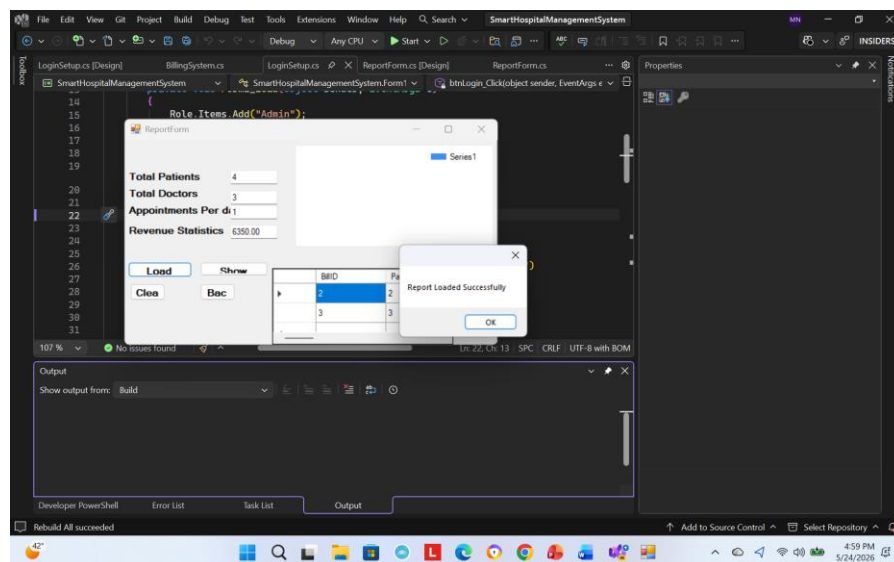
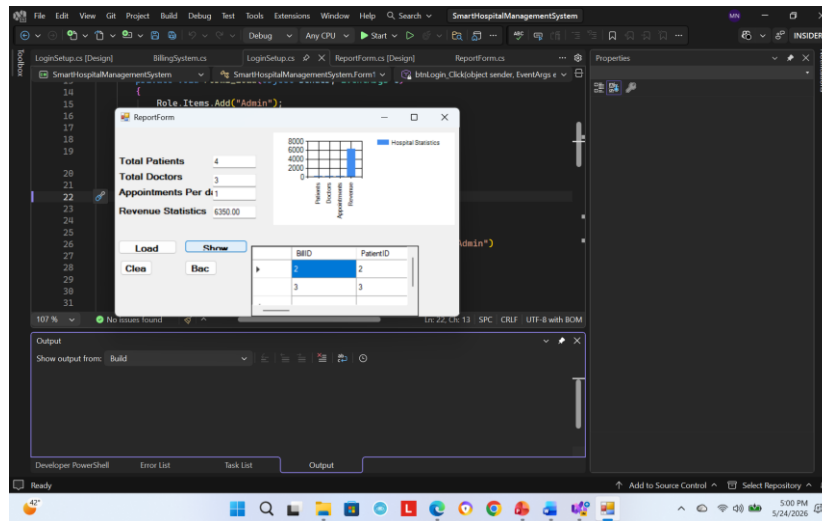


Figure Display Report and Chart

This screenshot shows graphical representation of hospital statistics using charts. The system successfully visualizes revenue and patient-related information for analysis and reporting purposes.



Challenges Faced During Development

During the development of the Smart Hospital Management System, several challenges were faced. One major challenge was connecting the C# Windows Forms application with the SQL Server database correctly. Different connection several query errors occurred during database operations such as insert, update, delete, and search.

Another challenge was designing the GUI forms in a clean and user-friendly way. Adjusting buttons, labels, textboxes, DataGridViews, and charts required time and repeated testing.

Handling validation was also difficult in the beginning. The system needed to properly manage invalid login credentials, empty fields, and incorrect data entries to avoid system crashes.

Managing multiple modules such as Patient Management, Appointment Scheduling, Billing, and Reports in one project also required proper organization of forms and database tables.

Handling backend coding behind every form was also a big challenge that require proper structure and syntax.

Despite these challenges, all modules were successfully implemented after testing and debugging.

Conclusion

The Smart Hospital Management System was successfully developed using C# Windows Forms and SQL Server Database. The project helps in managing important hospital operations such as patient records, appointments, billing, and reports in an organized way.

The system provides a simple and user-friendly interface that allows users to perform CRUD operations efficiently. Different modules were tested successfully, and the system produced accurate results during execution.

This project also improved understanding of GUI designing, database connectivity, SQL queries, form handling, and event-driven programming concepts in Visual Programming.

Overall, the project achieved its main objective of creating a computerized hospital management solution that saves time, reduces manual work, and improves record management.

References

1. Microsoft Documentation – Windows Forms
2. Microsoft SQL Server Documentation
3. Visual Studio 2022 IDE
4. C# Programming Language Concepts
5. SQL Query Language Tutorials
6. Course Lecture Notes and Lab Manuals
7. Online Resources from W3Schools and GeeksforGeeks
8. Online Recourses for handling backend coding and form designing